

1. 协议栈工作流程和无线收发控制 LED

实验内容

- 1)、ZigBee 协议栈简介
- 2)、如何使用 ZigBee 协议栈
- 3)、ZigBee 协议栈的安装、编译与下载
- 4)、协议栈无线收发控制 LED
- 5)、协议栈工作流程

实现现象:

协调器、终端上电，组网成功后 D1 灯闪烁

前言

前面讲了理论和实验都是为这章做铺垫的，整个学习中本章的实验是重中之重，也是我们以后实际开发中必须用到的。本套开发板以实战为主，用户可非常简单、方便的应用到实际产品中，本公司开发产品也是基于此开发板开发的，去掉了一些华而不实的东西，如 5 向按键、增加 OLED 显示、增加串口收发指示灯等等，用我们的开发板省去自己研究、裁剪没用的资源和代码，此项目工作涉及到软件、硬件、协议等知识，并非短时间就可调试好的。那些号称兼容 TI 程序的开发板，其实需要的水平最低，直接用 TI 给的参考设计，制板出来就行了；但是真正想应用到实际当中你就知道，自己裁剪的工作量是蛮大的。你说这年头几家公司用 5 向按键、黑白屏(而且是翻新的，他们没修改 Lcd 驱动，只能用 TI 提供的驱动)。以上只是有感而发，不说了，只希望您使用后能对我们的工作认可，我们知足了。

1. 介绍

ZigBee 无线传感器网络涉及电子、电路、通信、射频等多学科的知识，这对于入门级学习来说，无形中增加了学习难度，很多读者看协议、射频电路……学了半年甚至更长的时间，但是连基本的点对点通信都无法实现，更别说 ZigBee 网络应用了。基于此原因，本章采用一种新学习思路，快速帮大家理解、使用 ZigBee 协议栈。先进行 ZigBee 无线网络通信的学习和实验，有了感性认知后再看 ZigBee 协议栈视频、射频、天线等知识，看完后再实验一遍加深印象，用此法会起到事半功倍的效果，经过多名初学者学习所积累总结出的经验，希望对您有所帮助。

2. ZigBee 协议栈简介

什么是 ZigBee 协议栈呢？它和 ZigBee 协议有什么关系呢？协议是一系列的通信标准，通信双方需要共同按照这一标准进行正常的数据发射和接收。协议栈是协议的具体实现形式，通俗点来理解就是协议栈是协议和用户之间的一个接口，开发人员通过使用协议栈来使用这个协议的，进而实现无线数据收发。图 1 展示了 ZigBee 无线网络协议层的架构图。ZigBee 的协议分为两部分，IEEE 802.15.4 定义了 PHY（物理层）和 MAC（介质访问层）技术规范；ZigBee 联盟定义了 NWK（网络层）、APS（应用程序支持子层）、APL（应用层）技术规范。ZigBee 协议栈就是将各个层定义的协议都集合在一起，以函数的形式实现，并给用户提供 API(应用层)，用户可以直接调用。

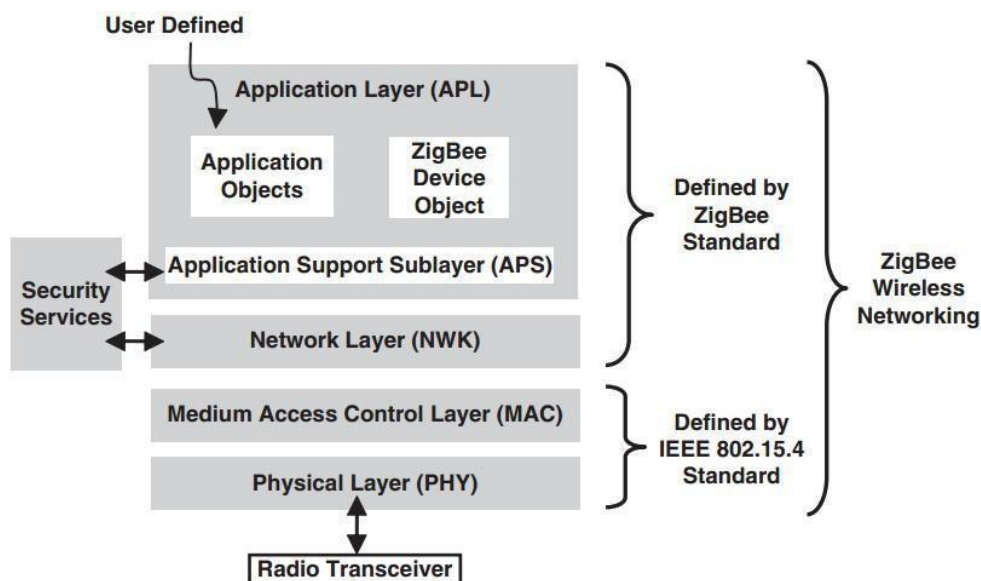


图 1 ZigBee 无线网络协议层

3. 如何使用 ZigBee 协议栈

协议栈是协议的实现，可以理解为代码，函数库，供上层应用调用，协议较底下的层与应用是相互独立的。商业化的协议栈就是给你写好了底层的代码，符合协议标准，提供给你一个功能模块给你调用。你需要关心的就是你的应用逻辑，数据从哪里到哪里，怎么存储，处理；还有系统里的设备之间的通信顺序什么的，当你的应用需要数据通信时，调用组网函数给你组建你想要的网络；当你想从一个设备发数据到另一个设备时，调用无线数据发送函数；当然，接收端就调用接收函数；当你的设备没事干的时候，你就调用睡眠函数；要干活的时候就调用唤醒函数。所以当你做具体应用时，不需要关心协议栈是怎么写的，里面的每条代码是什么意思。除非你要做协议研究。每个厂商的协议栈有区别，也就是函数名称和参数可能有区别，这个要看具体的例子、说明文档。

怎么使用 ZigBee 协议栈？举个例子，用户实现一个简单的无线数据通信时的一般步骤：

- 1、组网：调用协议栈的组网函数、加入网络函数，实现网络的建立与节点的加入。
- 2、发送：发送节点调用协议栈的无线数据发送函数，实现无线数据发送。
- 3、接收：接收节点调用协议栈的无线数据接收函数，实现无线数据接收。

是不是看上去很简单啊，其实协议栈很多都封装好了，下面我们大概看看无线发送函数：

```
1.  afStatus_t AF_DataRequest( afAddrType_t *dstAddr,
2.                               endPointDesc_t *srcEP,
3.                               uint16 cID,
4.                               uint16 len,
5.                               uint8 *buf,
6.                               uint8 *transID,
7.                               uint8 options,
8.                               uint8 radius )
```

用户调用该函数即可实现数据的无线数据的发送，此函数中有 8 个参数，用户需要将每个参数的含义理解以后，才能熟练使用该函数进行无线数据通信的目的。现在只讲其中最重要的两个参数，其它参数不需要死记硬背，以后用多了自然就记住了。

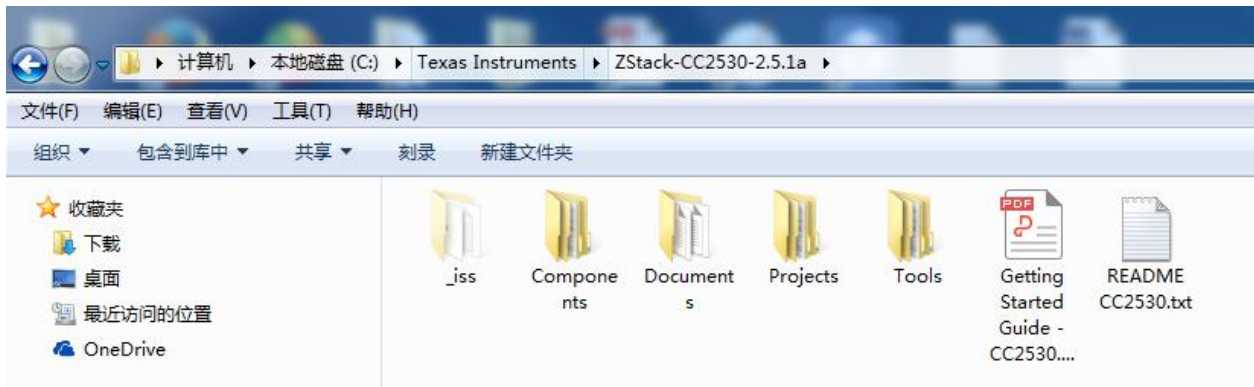
4. uint16 len, //发送数据的长度；
5. uint8 *buf, //指向存放发送数据的缓冲区的指针。

至于调用该函数后，如何初始化硬件进行数据发送等工作，用户不需要关心，ZigBee 协议栈已经将所需要的工作做好了，我们只需要调用相应的 API 函数即可，而不必关心具体实现细节。看起来是不是很简单呢，是不是有动手试试的冲动。先别急还要先安装 ZigBee 协议栈才能进行开发调试呢，下面就动手安装 ZigBee 协议栈吧。

4. ZigBee 协议栈的安装、编译与下载

解压 `..\开发软件\Zstack 2.5.1 协议栈安装文件.zip`；解压得到 .exe 文件，双击进行安装，路径你可以选择默认，同样你也可以选择你想要安装的位置。也许有人就困惑了，装完之后不是应该有个桌面图标么？其实所谓的安装协议栈只是把一些文件解压到你安装的目录下。好了，协议栈是安装好了，可是怎么用它呢？

装了肯定要用啊是吧，别急，我们先来看看这个协议栈的目录：



Components，顾名思义这个是放我们的库的文件夹，里面放了一些我们用到的 ZDO, driver, hal, zcl 等库的代码

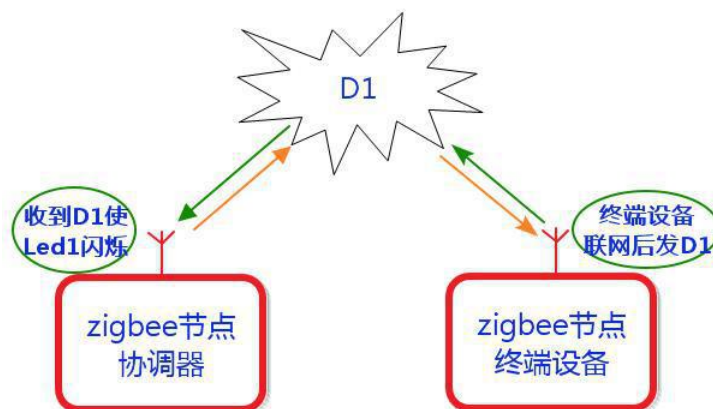
Documents，这个不用说大家都知道是放 TI 的开发文档的，里面很多都是讲述协议栈的 API 的有空时可以看看

Projects，这个文件夹放的是 TI 协议栈的例子程序，一个个例子程序都是以一个个 project 的形式给我们的，学好这些例子程序里面的一两个，基本你能做事情了。

Tools，这个文件夹是放 TI 的例子程序的一些上位机之类的程序，作为工具使用。

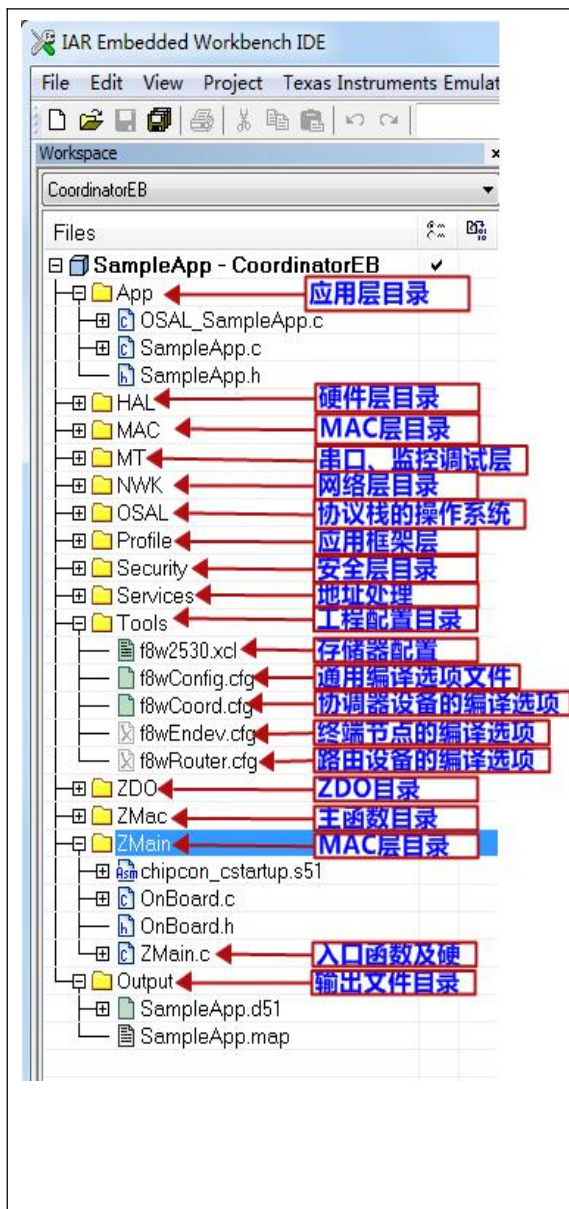
好了，基本明白了基本架构之后，我们以一个简单的实验开始。先掌握一点必要的理论再实验效果比较好。

5. 协议栈无线收发控制 LED



协议栈无线收发控制LED通讯过程

打开工程：[.\1. 协议栈工作流程和无线收发控制 LED\ZStack-2.5.1a\Projects\zstack\Samples\SampleApp\CC2530DB\SampleApp.eww](#)，从软件开发专业角度讲建议大家复制工程到非中文目录，因为有些开发环境对中文路径支持的不好，虽然 IAR 支持但在实际工作中你想别人看到你的工程，认为你很专业就照着上面做吧。我们演示就不修改，容易引起大家误会，打开工程如下图：



App: 应用层目录，这是用户创建各种不同工程的区域，在这个目录中包含了应用层的内容和这个项目的主要内容。

HAL: 硬件层目录，包含有与硬件相关的配置和驱动及操作函数。

MAC: MAC 层目录，包含了 MAC 层的参数配置文件及其 MAC 的 LIB 库的函数接口文件。

MT: 实现通过串口可控制各层，并与各层进行直接交互

NWK: 网络层目录，包含网络层配置参数文件网络层库的函数接口文件及 APS 层库的函数接口。

OSAL: 协议栈的操作系统。

Profile: Application framework 应用框架层目录，包含 AF 层处理函数文件。应用框架层是应用程序和 APS 层的无线数据接口。

Security: 安全层目录，包含安全层处理函数，比如加密函数等

Services: 地址处理函数目录，包括地址模式的定义及地址处理函数。

Tools: 工程配置目录，包括空间划分及 Z-Stack 相关配置信息。

ZDO: ZDO 目录

ZMac: MAC 层目录，包括 MAC 层参数配置及 MAC 层 LIB 库函数回调处理函数。

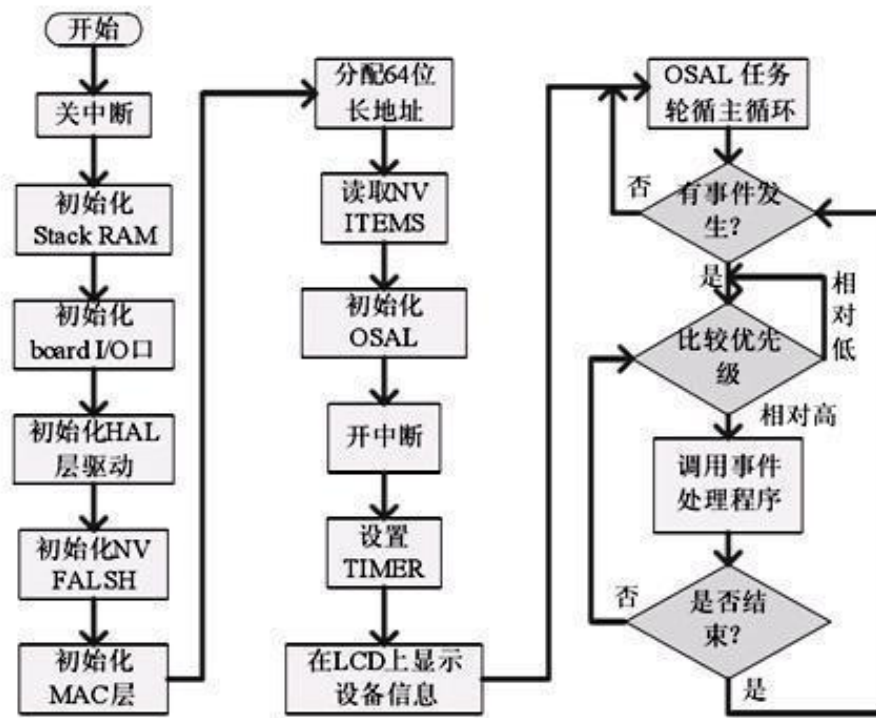
ZMain: 主函数目录，包括入口函数及硬件配置文件。

Output: 输出文件目录，由 IAR IDE 自动生成。

带协议栈的工程怎么这么多文件夹和文件，都有什么用啊？现阶段大家只要带着这个疑问照着做实验就行了，后面实验接触多了自然就懂了。

下面我就带着大家分析协议栈工作流程。

(1) zigbee 工作流程



看源码推荐大家使用 Source_Insight，十分强大的工具，从事软件必备软件，除非你想做菜鸟，具体软件使用请参考“\开发软件\Source Insight 3.5.rar”。下面列出实验中涉及到比较重要的函数进行详解，由于是带协议栈第一个实验，我们对源码也进行注释，方便习惯看源码的同志学习。我建议大家先看看下面的文章，再阅读一次源码加深印象，后面的例子结构基本相同，所以学好此实验，再做后面的实验就得得心应手了。用户自己添加的应用任务程序在 Zstack 中的调用过程：

main()---> osal_init_system()---> osalInitTasks()---> SampleApp_Init()

下面我们就先从 main()函数开始吧。

提示：如果你第一次接触 ZStack，第一个实验的代码看注释只须大概知道它们是做什么的，有点印象就行了，后面实验会徐徐渐进，慢慢带领大家搞懂整个流程和代码的。如果刚开始就啃代码，不但效率低而且信心受损。

(2) 程序入口

打开 ZMain.c 找到 main 函数

```

int main( void )
{
    osal_int_disable( INTS_ALL ); //关闭所有中断
    HAL_BOARD_INIT();             //初始化系统时钟
    zmain_vdd_check();            //检查芯片电压是否正常
    InitBoard( OB_COLD );         //初始化 I/O，LED、Timer 等
    HalDriverInit();              //初始化芯片各硬件模块
    osal_nv_init( NULL );         //初始化 Flash 存储器
    ZMacInit();                   //初始化 MAC 层
    zmain_ext_addr();             //确定 IEEE 64 位地址
    zgInit();                     //初始化非易失变量

    #ifndef NONWK
        // Since the AF isn't a task, call it's initialization routine
    
```

```

afInit();
#endif

osal_init_system();           //初始化操作系统
osal_int_enable( INTS_ALL ); //使能全部中断
InitBoard( OB_READY );       //最终板载初始化
zmain_dev_info();            //显示设备信息
#ifdef LCD_SUPPORTED
zmain_lcd_init();            //初始化 LCD
#endif
#ifdef WDT_IN_PM1
/* If WDT is used, this is a good place to enable it. */
WatchDogEnable( WDTIMX );
#endif

osal_start_system(); // No Return from here 执行操作系统，进去后不会返回
return 0; // Shouldn't get here.
} // main()

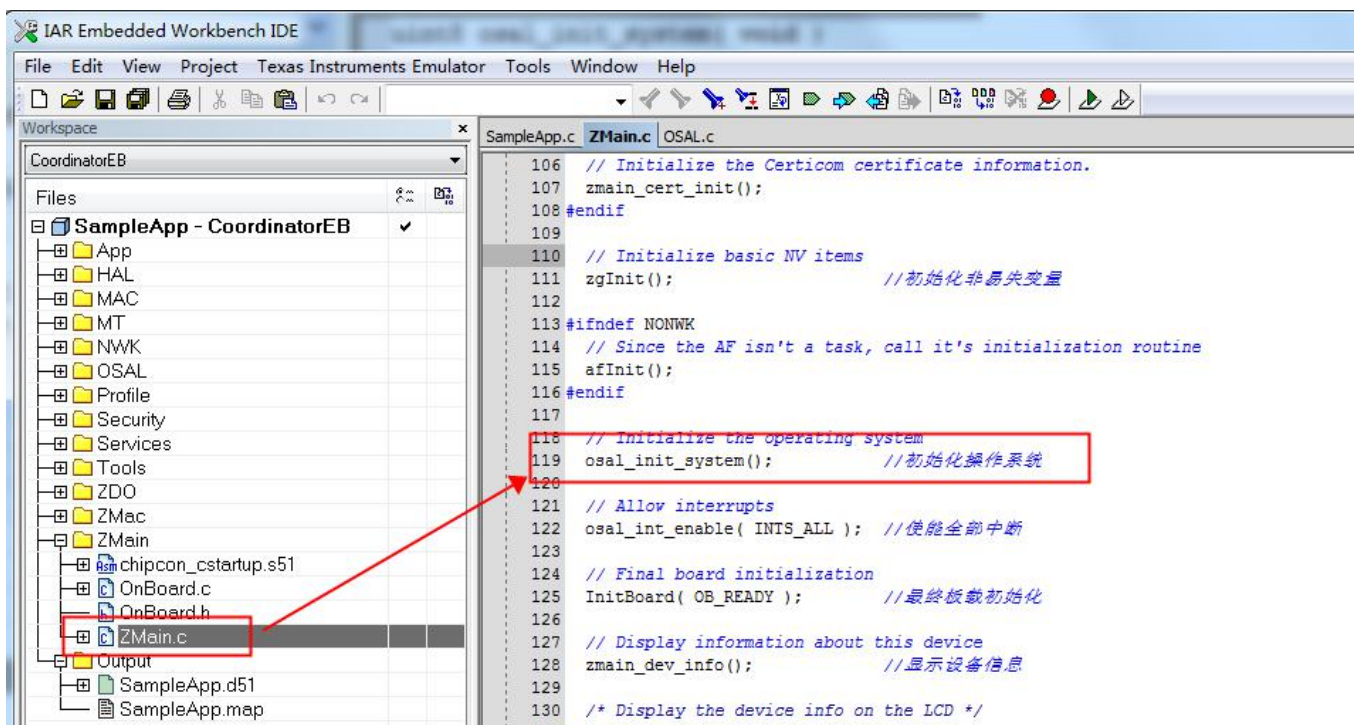
```

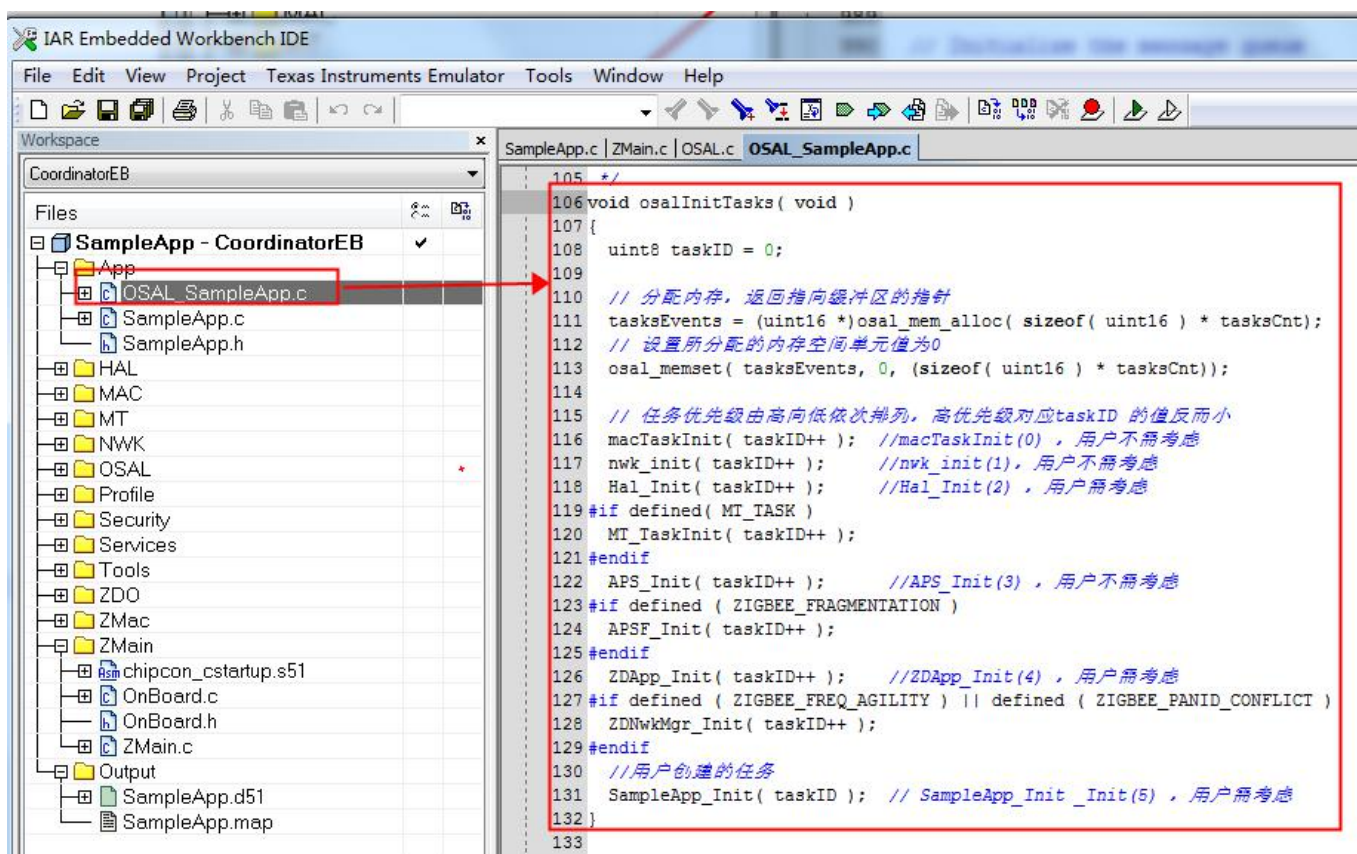
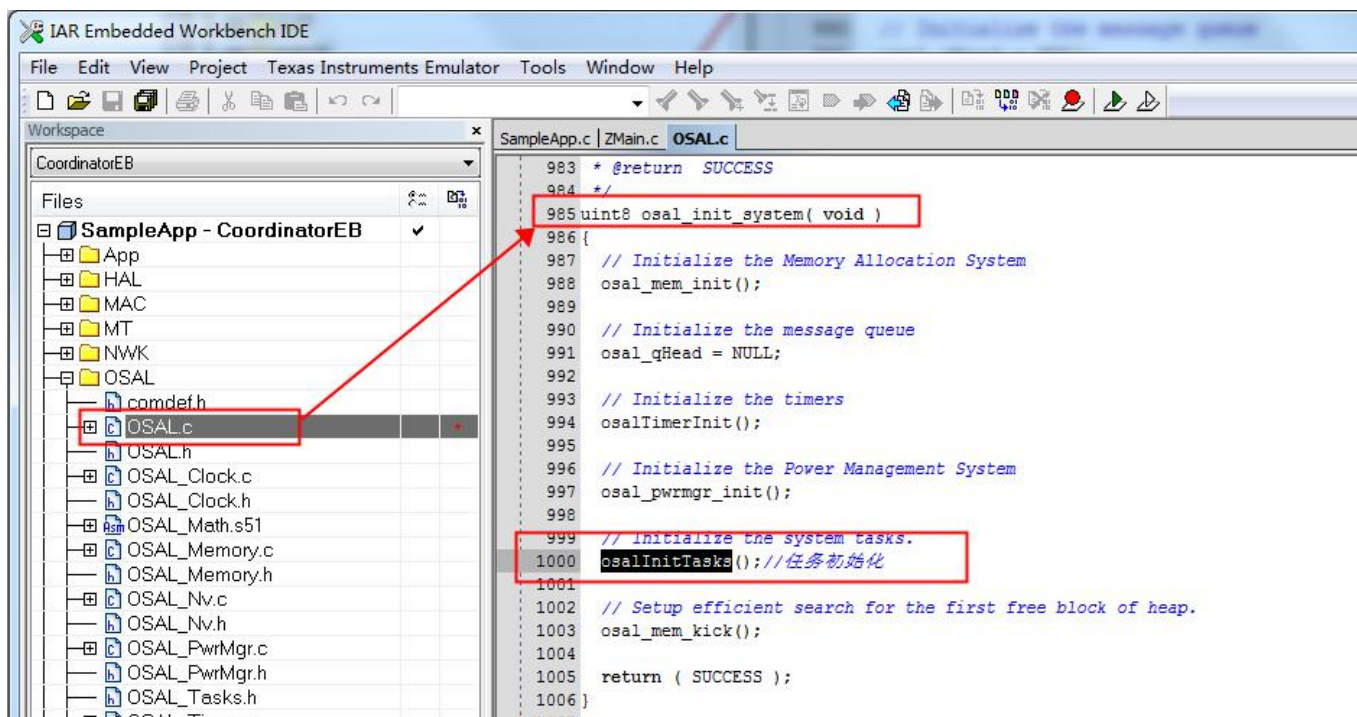
看了上面的代码后，可能感觉很多函数不认识。没关系刚开始大概了解流程即可，main 函数先执行初始化工作，包括硬件、网络层、任务等的初始化。然后执行 `osal_start_system()`；操作系统，进去后可不会回来了。在这里，我们重点了解 2 个函数：

初始化操作系统 `osal_init_system()`；
运行操作系统 `osal_start_system()`；

(3) 系统初始化

先来看 `osal_init_system()`；系统初始化函数，进入函数。如果用 IAR 看代码可在函数名上单击右键 --> go to definition of...，便可以进入函数。发现里面有 6 个初始化函数，这里我们只关心 `osalInitTasks()`；任务初始化函数，继续由该函数进入。





函数对 taskID 进行初始化, 每初始化一个 taskID++。大家看到了注释后面有些写着用户需要考虑, 有些则写着用户不需考虑。没错, 需要考虑的用户可以根据自己的硬件平台或者其他设置, 而写着不需考虑的也是不能修改的。SampleApp_Init() 是我们应用协议栈例程的必要函数, 用户通常在这里初始化自己的东西。至此, osal_init_system(); 大概了解完毕。

(4) 运行操作系统

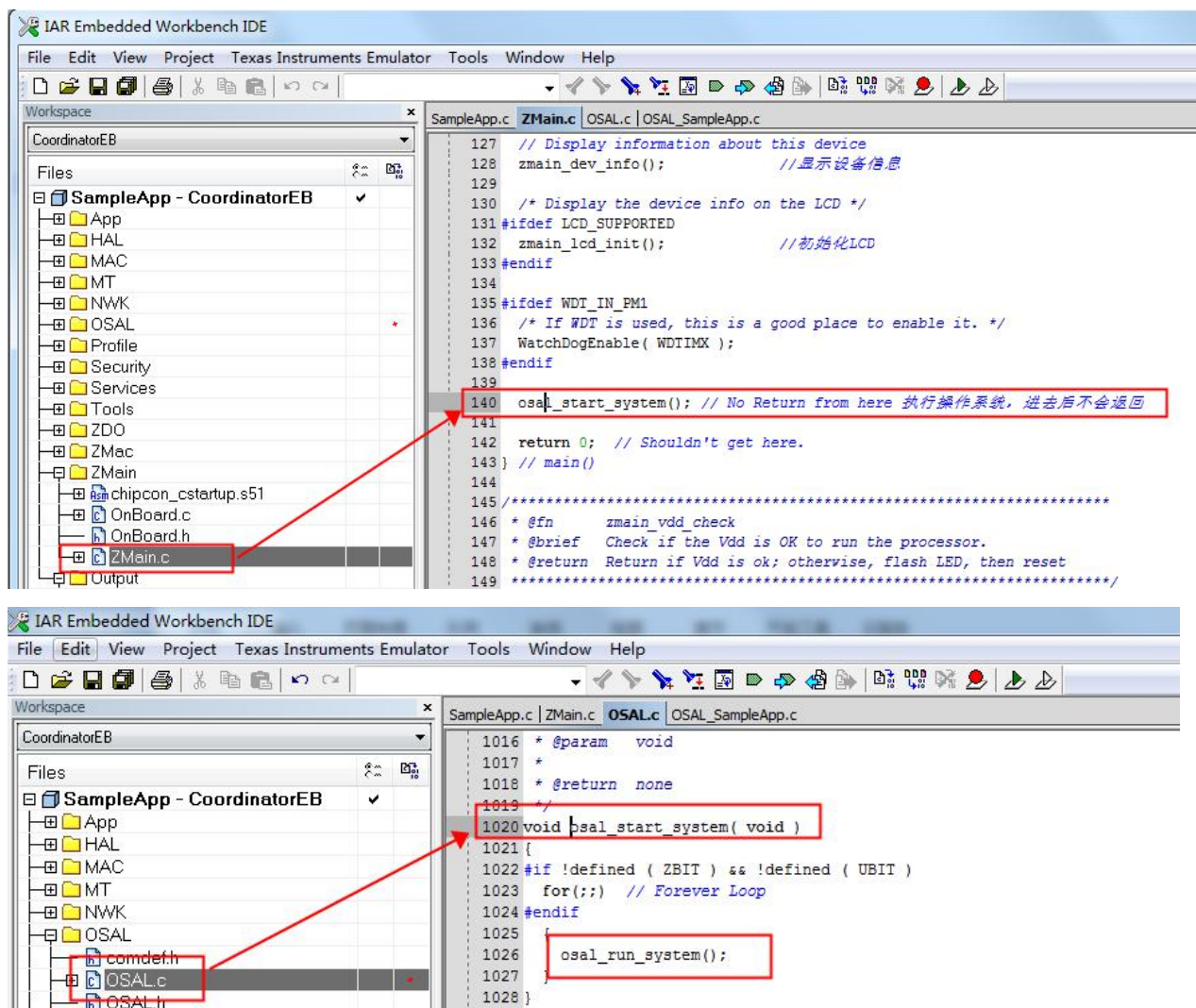
深圳市亿研电子有限公司

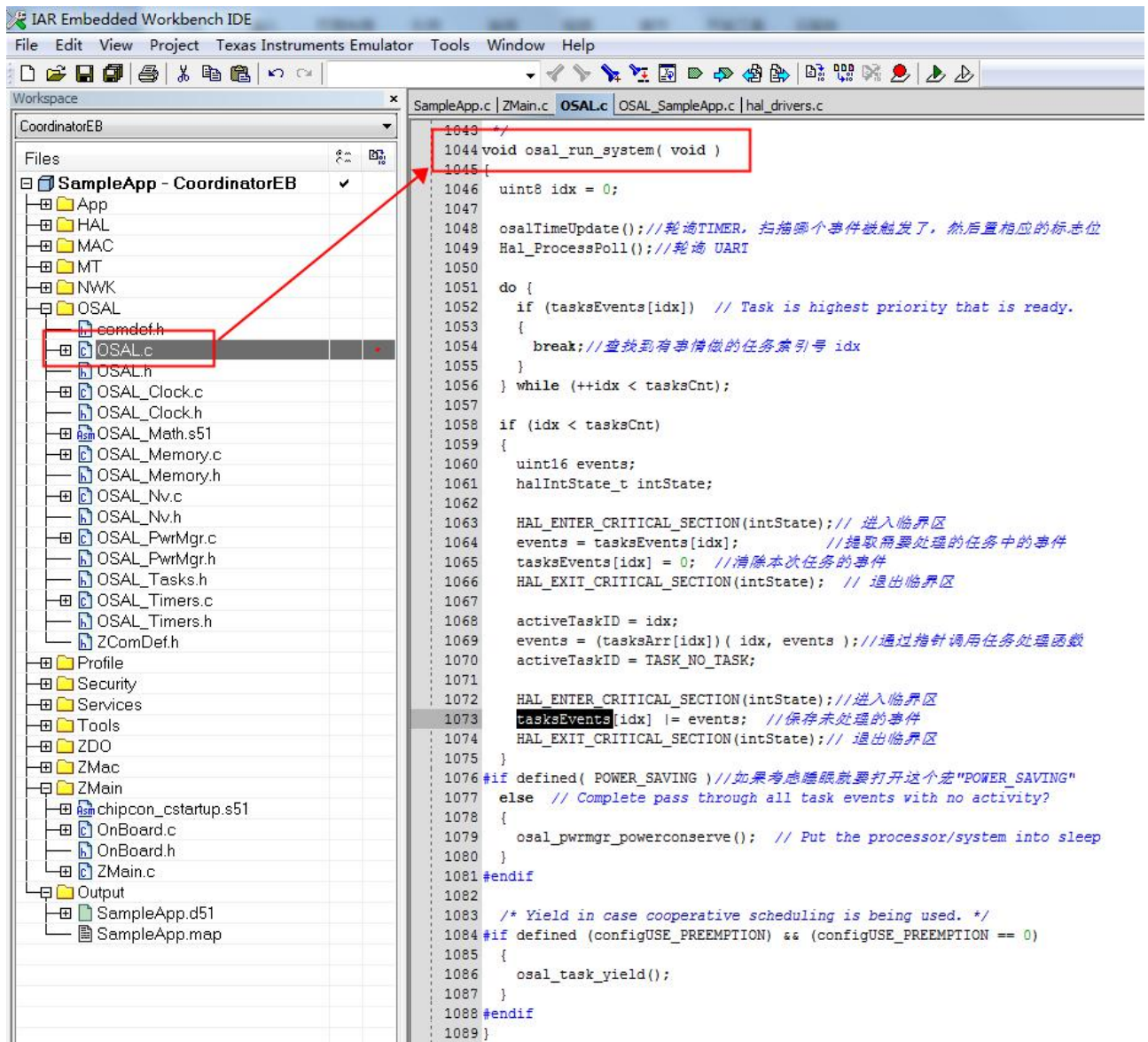
网址: <http://sz-vv.taobao.com/>

技术交流 QQ:89339784 咨询客服获取最新 QQ 群

售后电话:18682359439

接下来看第二个函数 `osal_start_system()`; 运行操作系统。同样用 go to definition 的方法进入该函数。

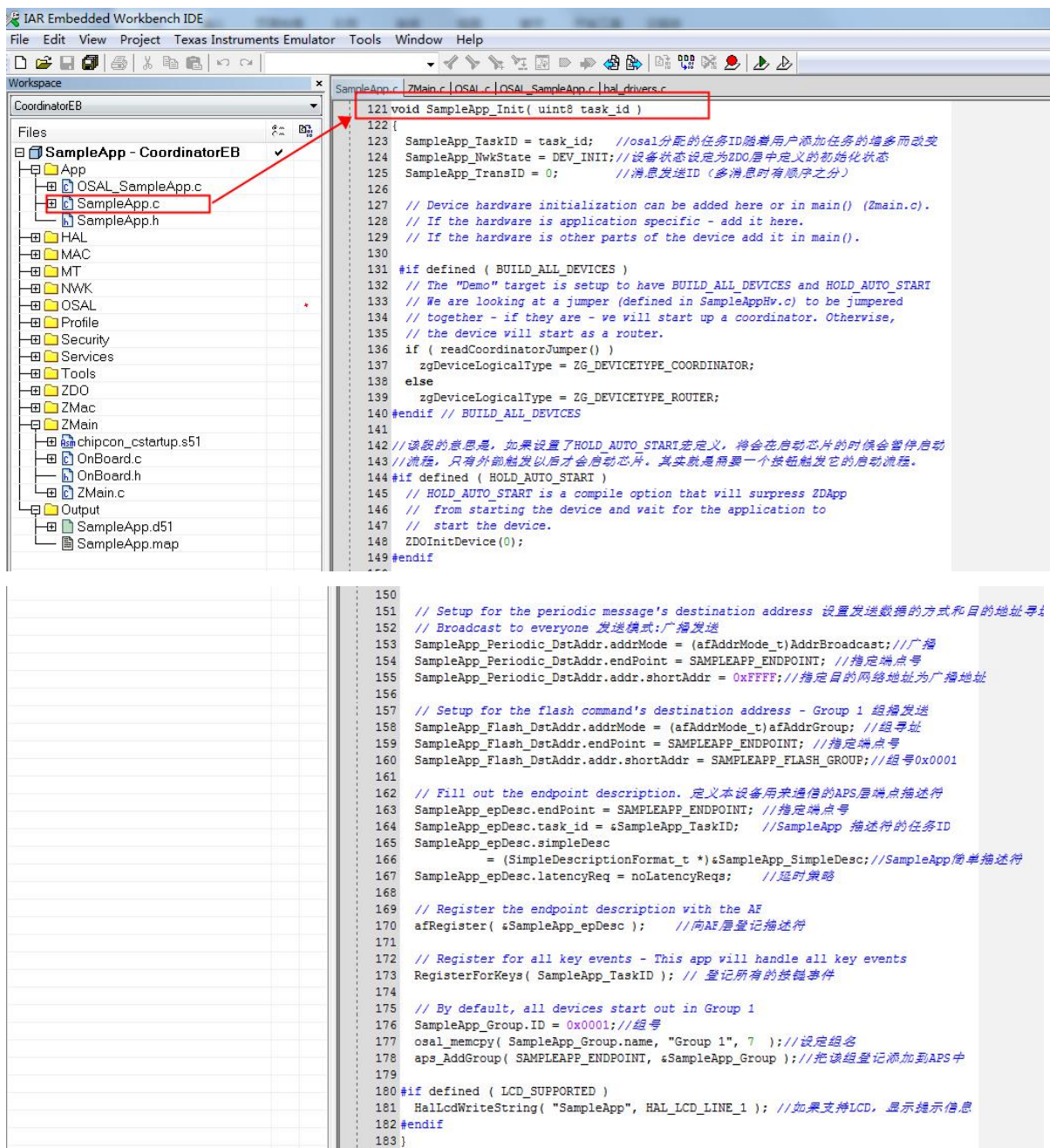




我们看一下 `events = tasksEvents[idx];` 进入 `tasksEvents[idx]` 数组定义，发现恰好是 `osalInitTasks()` 函数里面分配空间初始化过的 `tasksEvents`。而且 `taskID` 一一对应。这就是初始化与调用的关系。`taskID` 把任务联系起来了。

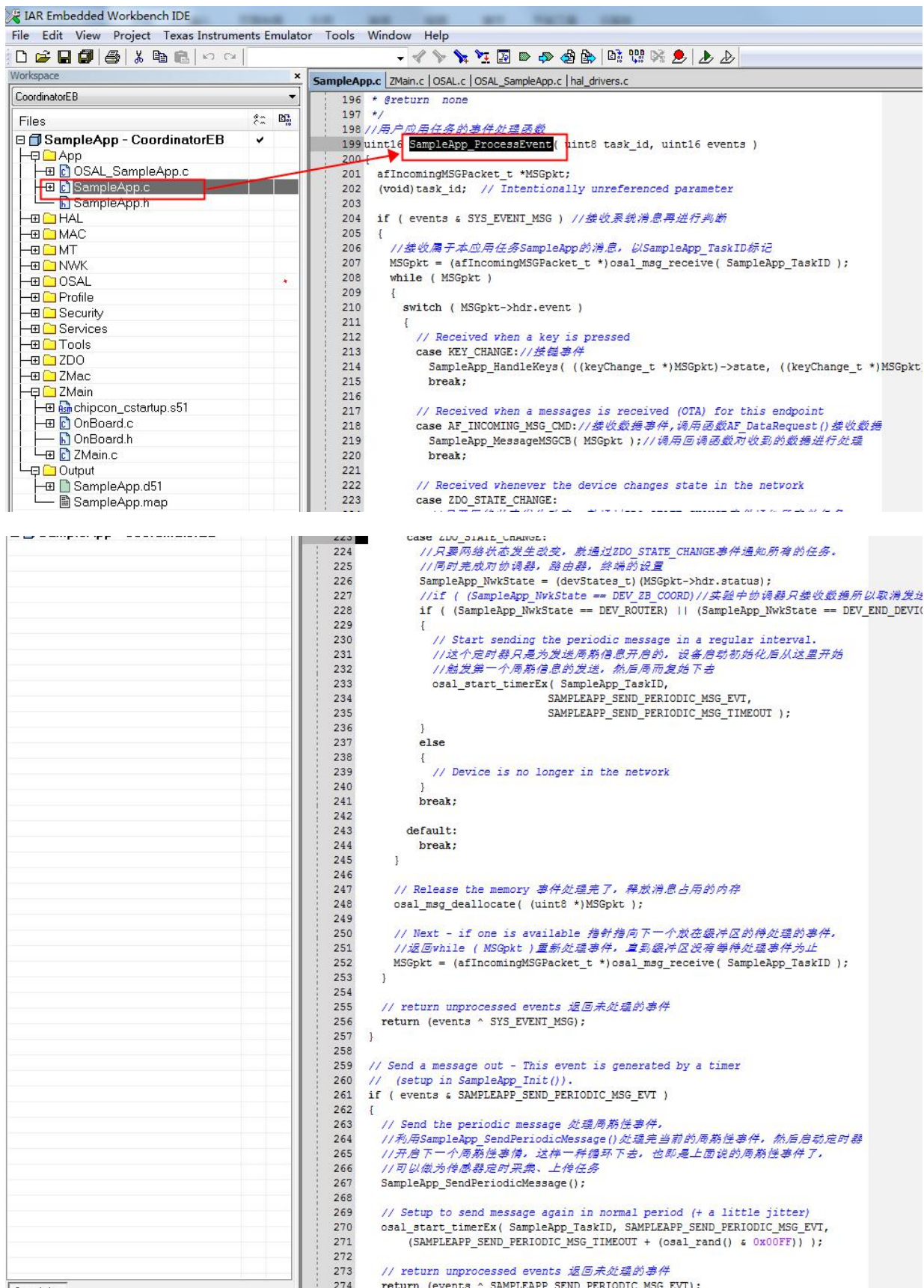
(5) 用户应用任务初始化

函数 `SampleApp_Init()` 是用户应用任务初始化函数。



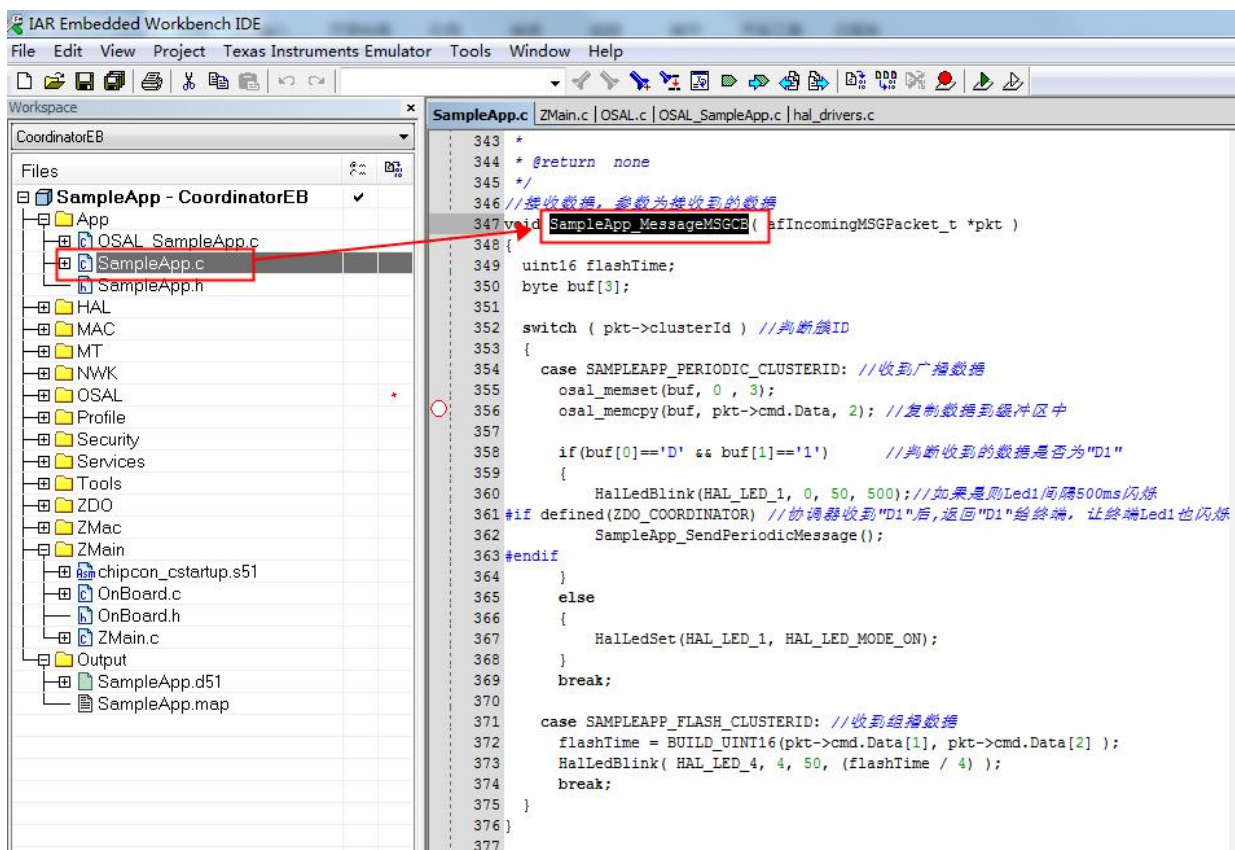
(6) 应用任务事件处理函数

SampleApp_ProcessEvent() 用户应用任务的事件处理函数。



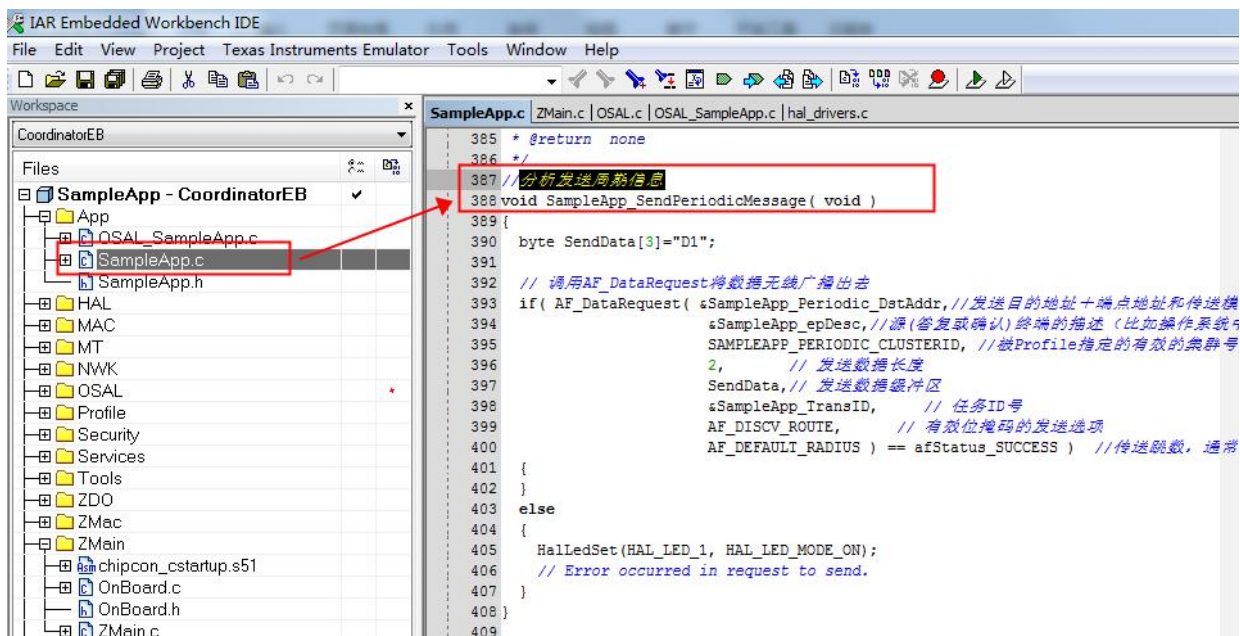
(7) 应用层无线消息处理

应用层的无线消息处理是在函数 SampleApp_MessageMSGCB() 里。



(8) 周期发送数据

周期发送信息 SampleApp_SendPeriodicMessage() 函数周期发送了“D1”，内部调用了函数 AF_DataRequest 进行无线消息发送，请注意看注释。

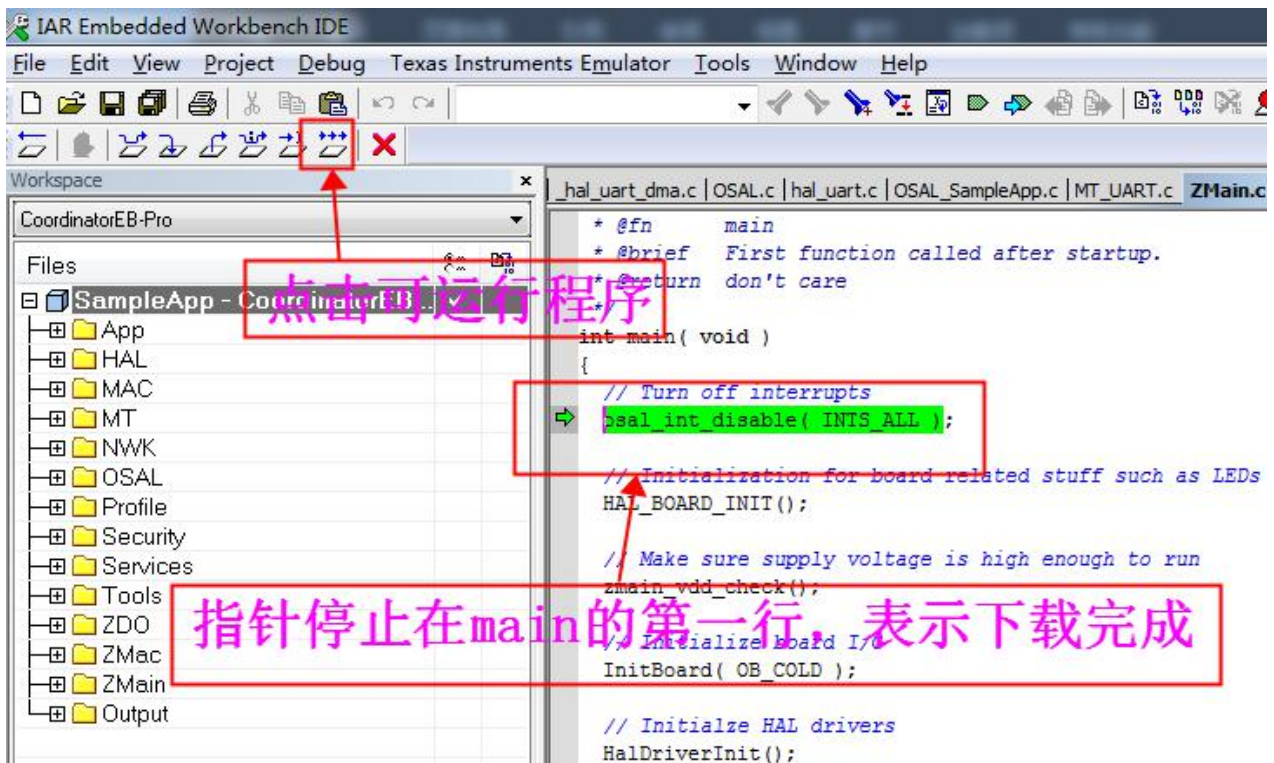
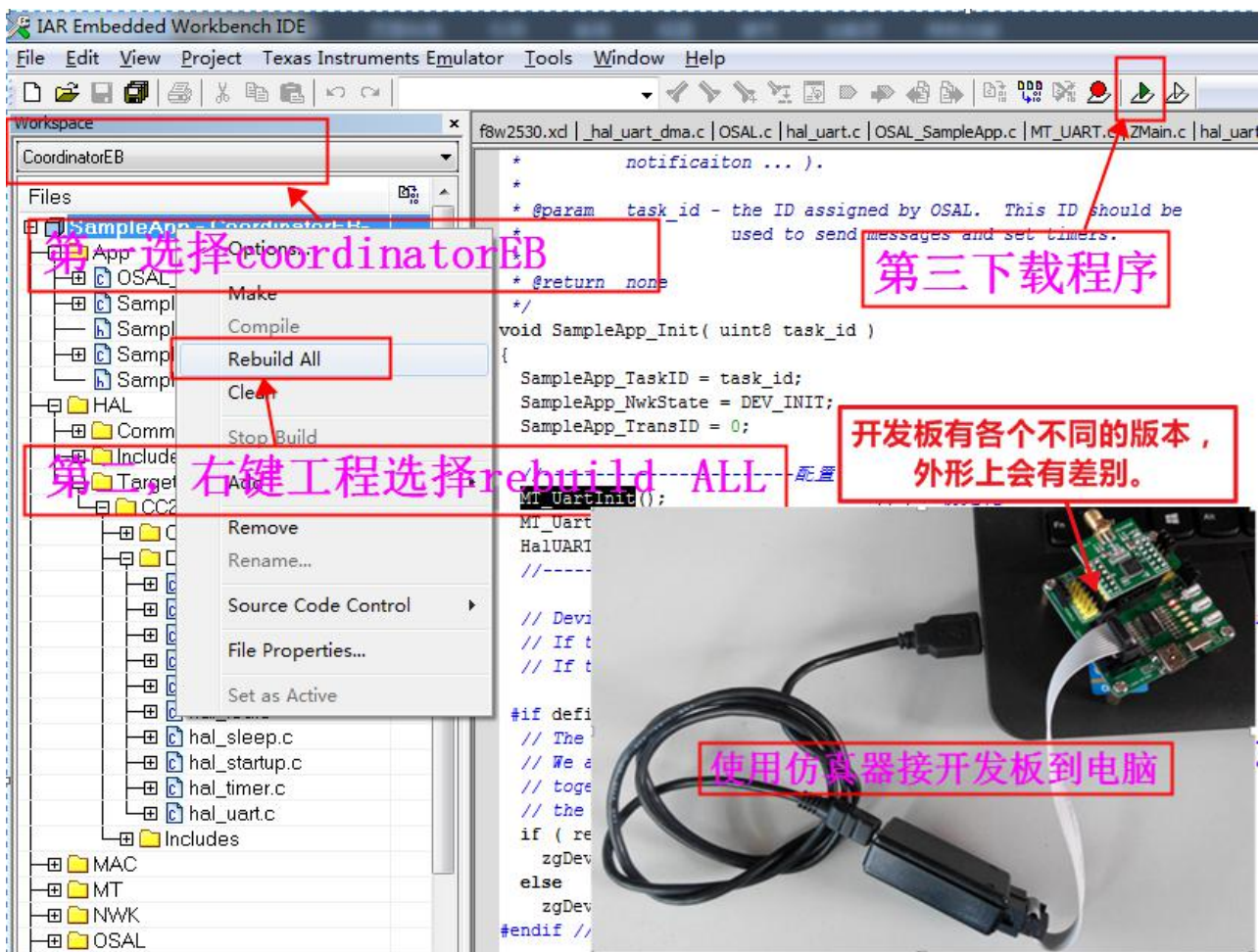


好了，第一次就讲这么多吧，内容很多但非常重要，最好理解后再去做后面的实验，打好坚实的基础后，再看后面的实验相对容易很多。

建议文档和实验目录下的代码一起对比看，注释非常重要！！

(9) 实验步骤

① 选择 CoordinatorEB (作为协调器), 下载到开发板 A(协调器), 如:



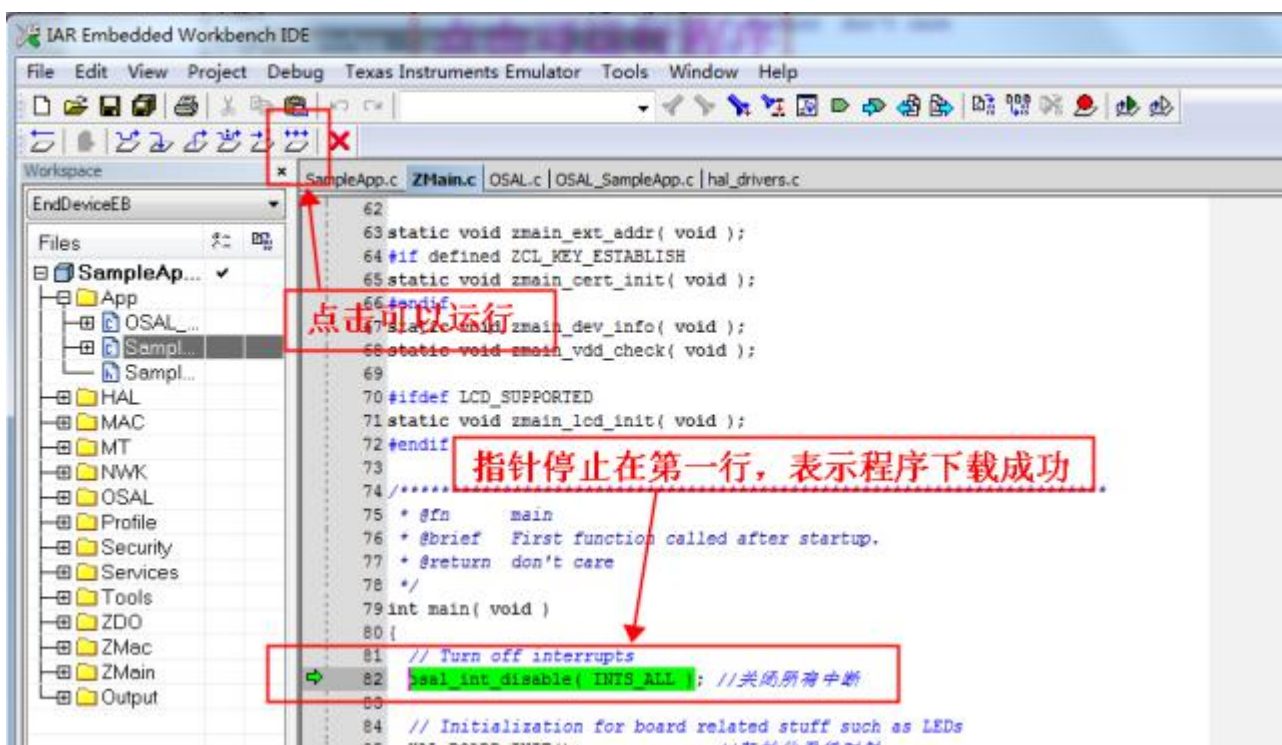
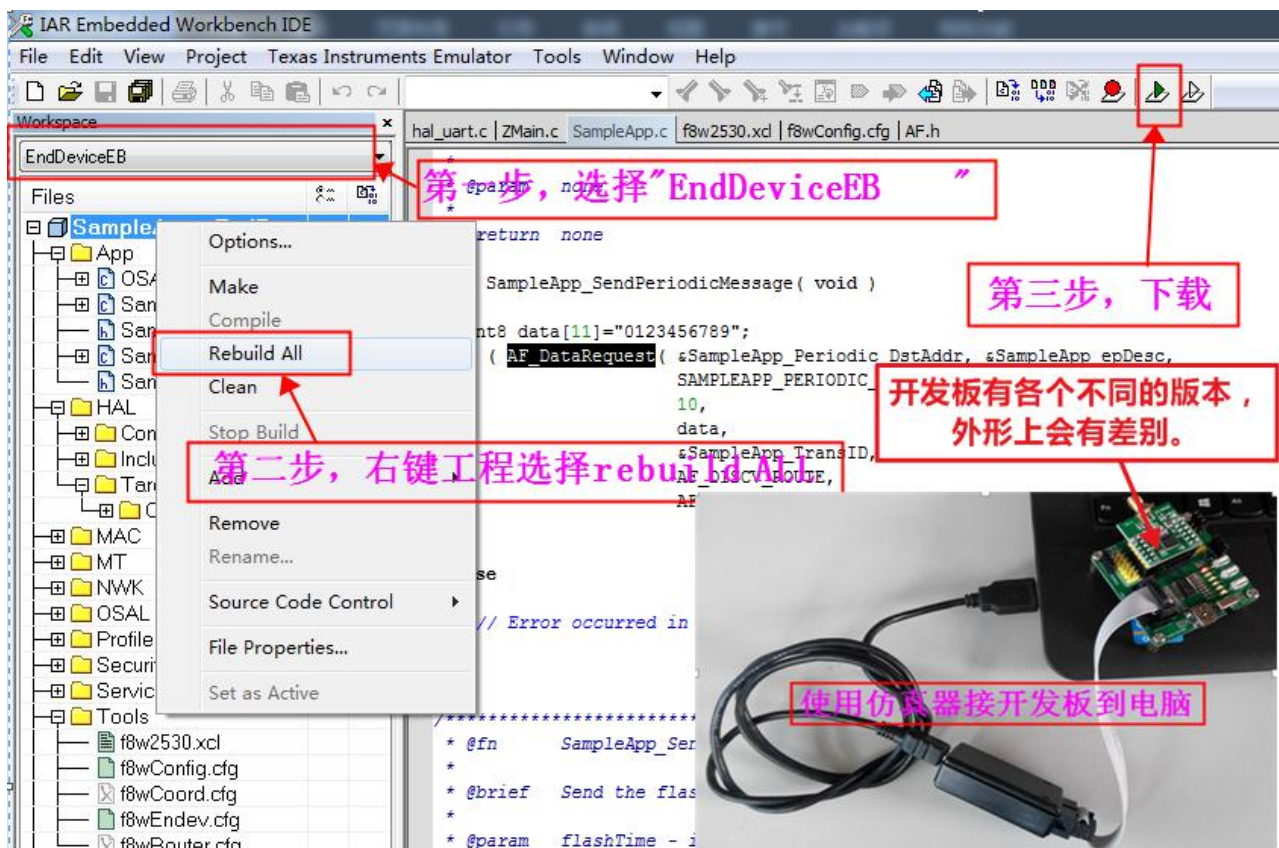
② 选择 EndDeviceEB (终端), 下载到开发板 B, 如:

深圳市亿研电子有限公司

网址: <http://sz-vv.taobao.com/>

技术交流 QQ:89339784 咨询客服获取最新 QQ 群

售后电话:18682359439



- ③ 给两块开发板上电，通过观察 D1 来判断组网是否成功
- ④ 当终端 D1 常亮后，紧接着变成闪灯状态，就是这个实验的现象。
- ⑤ 事实上，当终端组网成功后，就开始发送“D1”字符给协调器，协调器收到后，D1 开始闪灯，并返回“D1”给终端；终端收到“D1”后，D1 开始闪灯，并发送“D1”给协调器，周而复始。。。

注：此实验不用修改任何 option 里的配置，直接解压出来即可以编译下载，谢谢。

深圳市亿研电子有限公司

网址：<http://sz-vy.taobao.com/>

技术交流 QQ:89339784 咨询客服获取最新 QQ 群

售后电话:18682359439